Applying Set Reconciliation to Lightning Gossip

Jonathan Harvey-Buschel, Chaincode Labs

MIT DCI, September 2026

Overview

- Review: LN Gossip
- Previous Work
- Current network status
- Review: Set Reconciliation
- Gossip v1.5 & Proposal
- Known Problems, Future Work

Review: LN Gossip

Lightning Network: Payment channels + P2P connections + **Gossip**

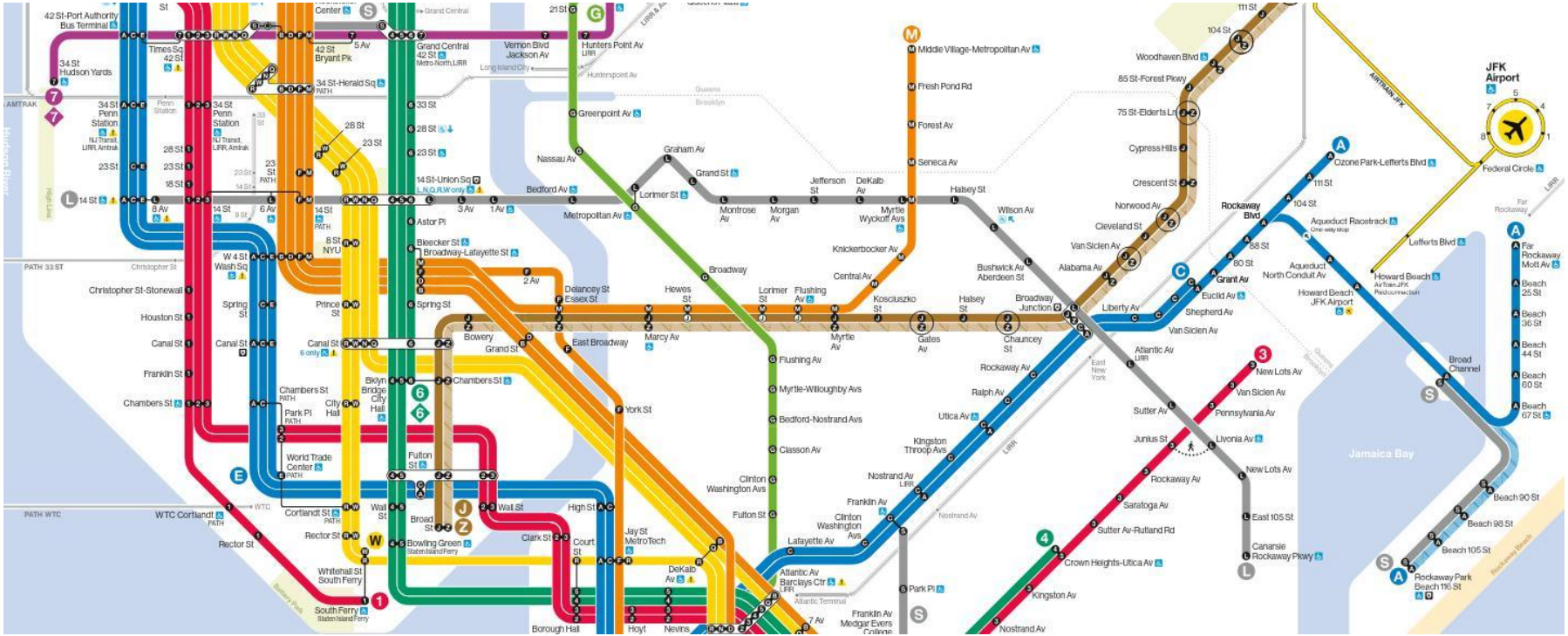
Review: LN Gossip

Lightning Network: Payment channels + P2P connections + **Gossip**

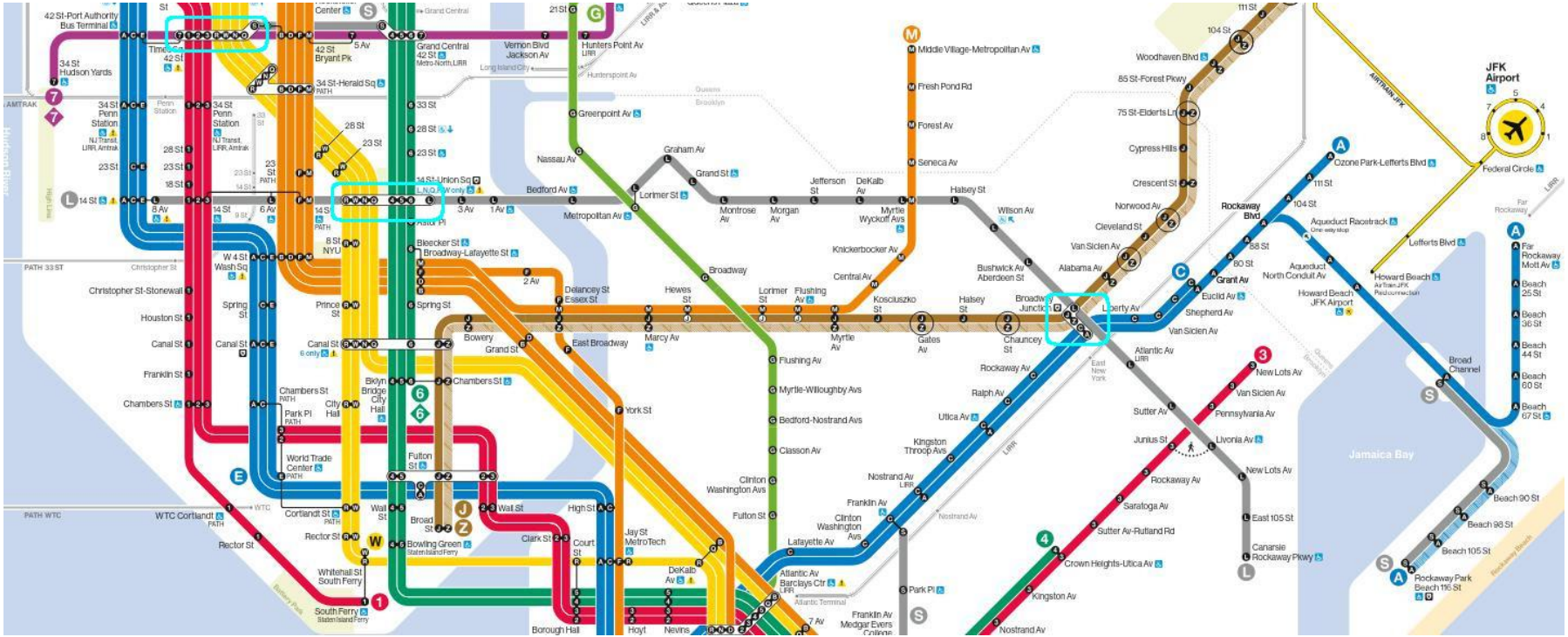
Gossip: All nodes are aware of each other, and all channels

Aware: IP addresses, channel feerates & capacity, etc.

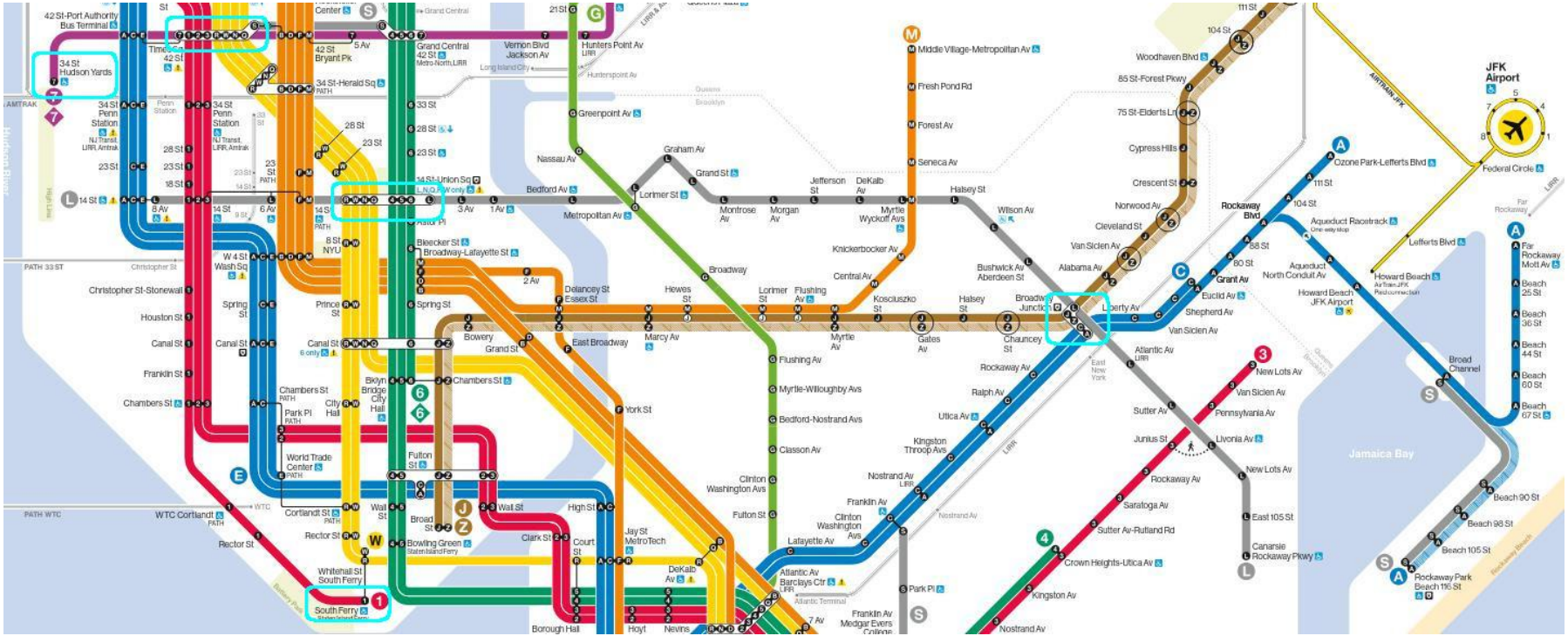
Review: LN Gossip



Review: LN Gossip

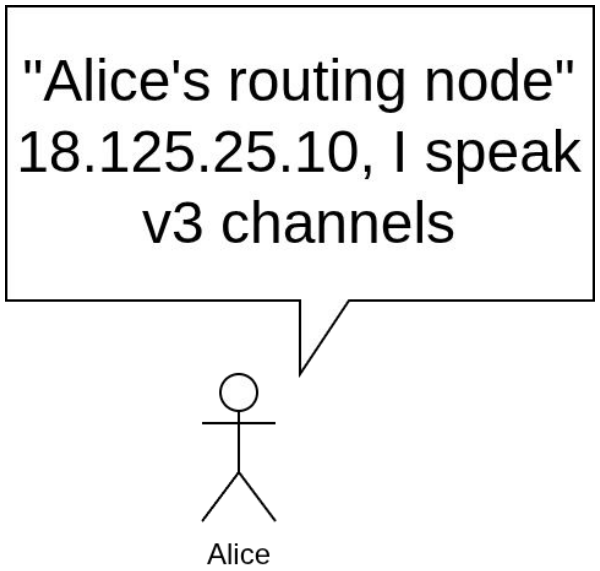


Review: LN Gossip



Review: LN Gossip

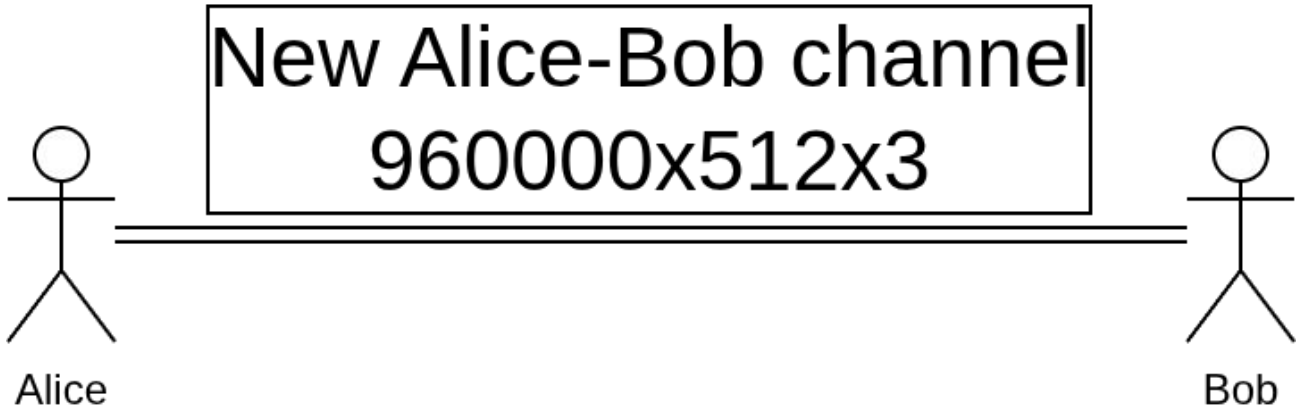
Node announcement: Net addresses, features, alias



Review: LN Gossip

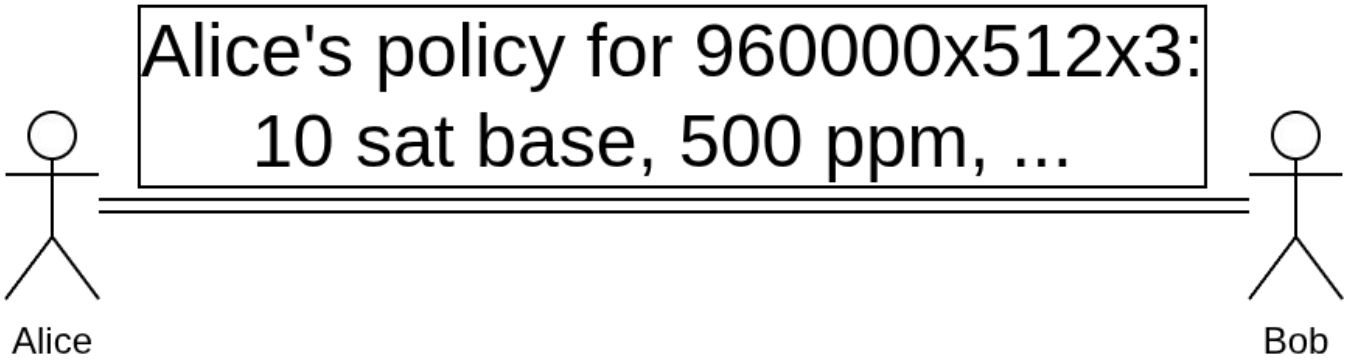
Channel announcement: Countersigned pointer to a UTXO

Short channel ID (SCID): blockheight + txindex + output index

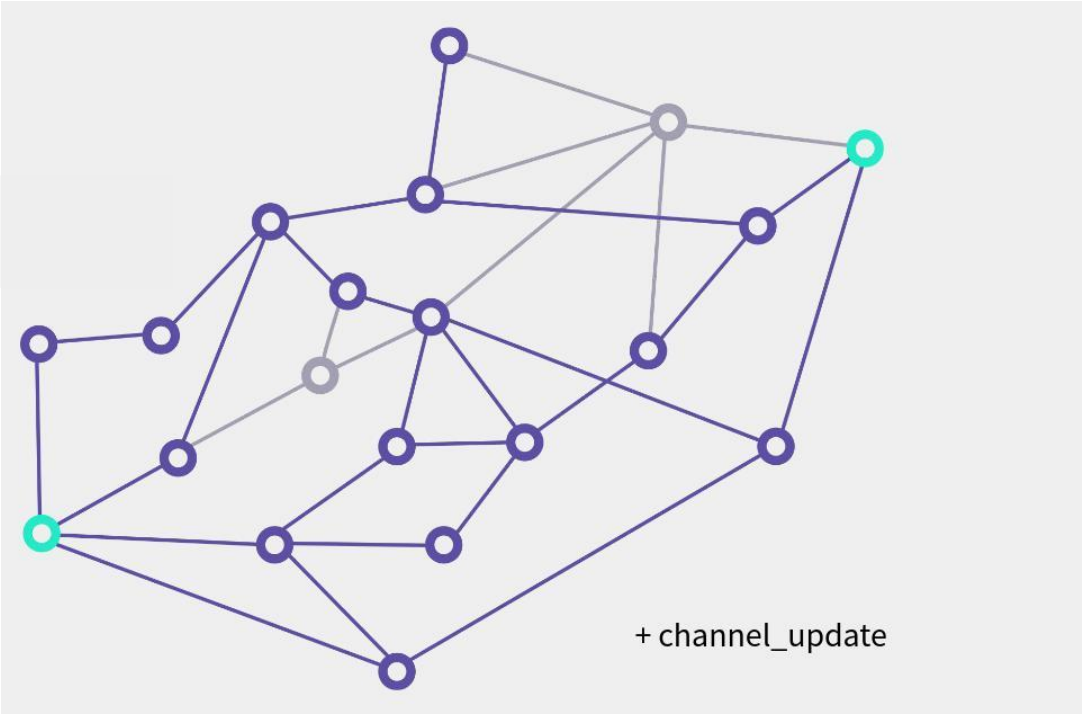


Review: LN Gossip

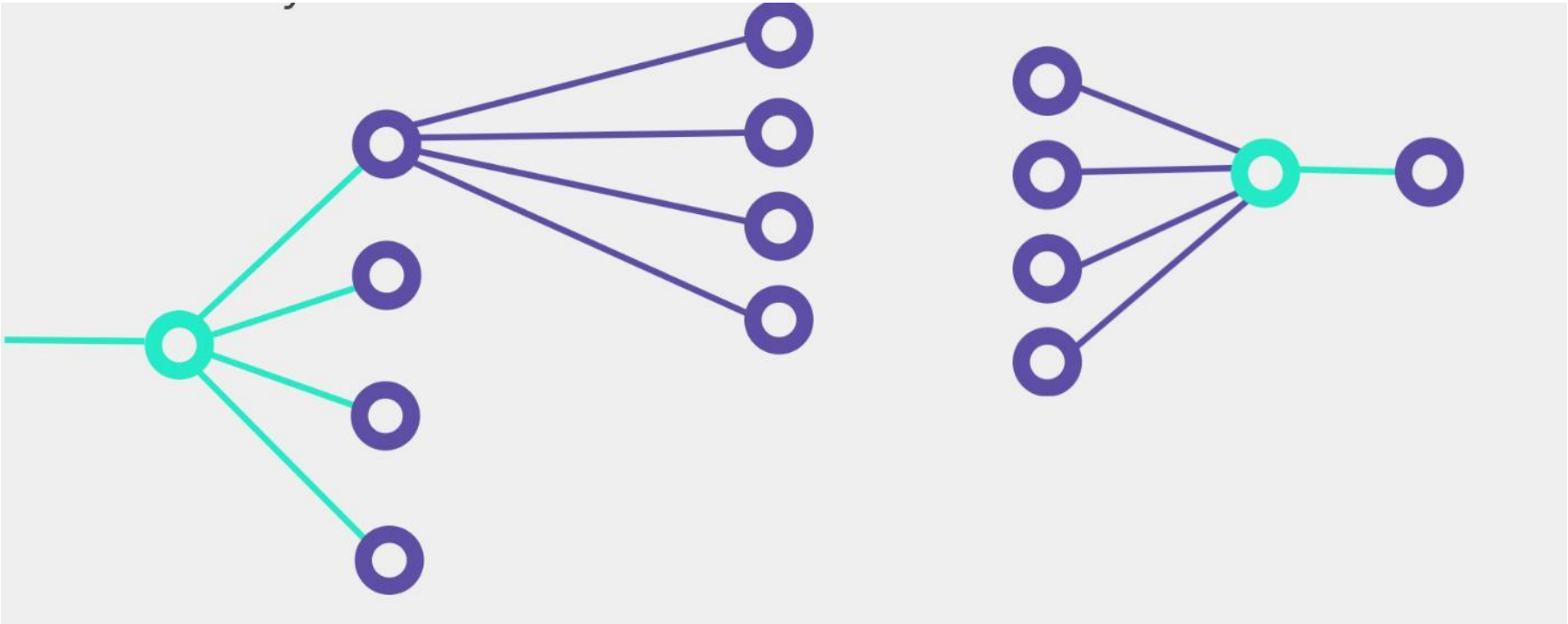
Channel update: One counterparty changes fees, min/max payment size, etc.



Review: LN Gossip



Review: LN Gossip



Review: LN Gossip

What if gossip fails?

Review: LN Gossip

What if gossip fails?

Failed payments, overpayment

Worse routing

Review: LN Gossip

Does it work now?

```
// The lightning network's gossip sync system is completely broken in numerous ways.
//
// Given no broadly-available set-reconciliation protocol, the only reasonable approach is
// to do a full sync from the first few peers we connect to, and then receive gossip
// updates from all our peers normally.
//
// Originally, we could simply tell a peer to dump us the entire gossip table on startup,
// wasting lots of bandwidth but ensuring we have the full network graph. After the initial
// dump peers would always send gossip and we'd stay up-to-date with whatever our peer has
// seen.
//
```

rust-lightning, routing/gossip.rs:L792

Review: LN Gossip

Does it work now?

- “I think our channel updates aren’t propagating”
- “DoS-like” behavior with (serving) gossip queries
- “Inbound gossip volume caused performance issues; we turned it off”
- “Our channel_announcement never propagated to our other nodes”

Across implementations & node sizes

Previous Work: Metrics

- https://github.com/lnresearch/topology
  - Archives of gossip from multiple Core Lightning nodes, 2020-2023
- https://github.com/lnresearch/gossip
  - New gossip archives, starting from early 2025

- Total traffic, graph snapshots, rate-of-change

Previous Work: Proposals

On the Routing Convergence Delay in the Lightning Network

Niklas Gögge[1], Elias Rohrer[2], and Florian Tschorsch[2]

[1] Distributed Security Infrastructures
Technical University of Berlin
n.goegge@campus.tu-berlin.de
[2] Distributed Security Infrastructures
Technical University of Berlin
{elias.rohrer, florian.tschorsch}@tu-berlin.de

Scaling the Lightning Network
with Practical Set Reconciliation*

Xingyu Chen *Boston University*
Boston, USA
chxy517@bu.edu

Anish Sinha *Red Hat*
Boston, USA
anishs@bu.edu

David Starobinski *Boston University*
Boston, USA
staro@bu.edu

Ari Trachtenberg *Boston University*
Boston, USA
trachten@bu.edu

June 18, 2026

Previous Work: Erlay

Bandwidth-Efficient Transaction Relay in Bitcoin

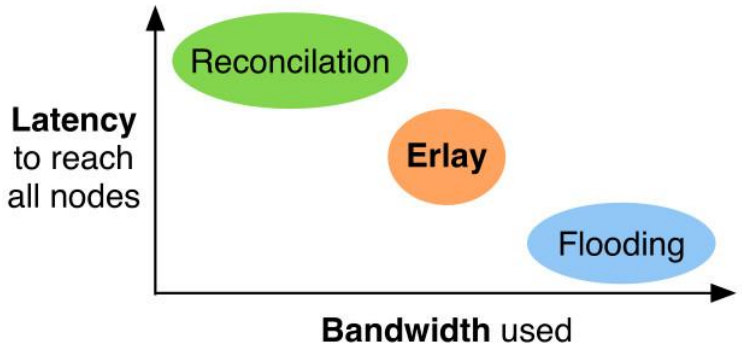
Gleb Naumenko
naumenko.gs@gmail.com
University of British Columbia

Gregory Maxwell
greg@xiph.org

Pieter Wuille
pwuille@blockstream.com
Blockstream

Alexandra Fedorova
sasha@ece.ubc.ca
University of British Columbia

Ivan Beschastnikh
bestchai@cs.ubc.ca
University of British Columbia



Current network status: Metrics

- Convergence Delay: Time for **X%** of nodes to receive a message
- Size: **Unique** bytes received
- Bandwidth: Bytes sent/received over time
- Frequency: How often we observe a message for a SCID or node

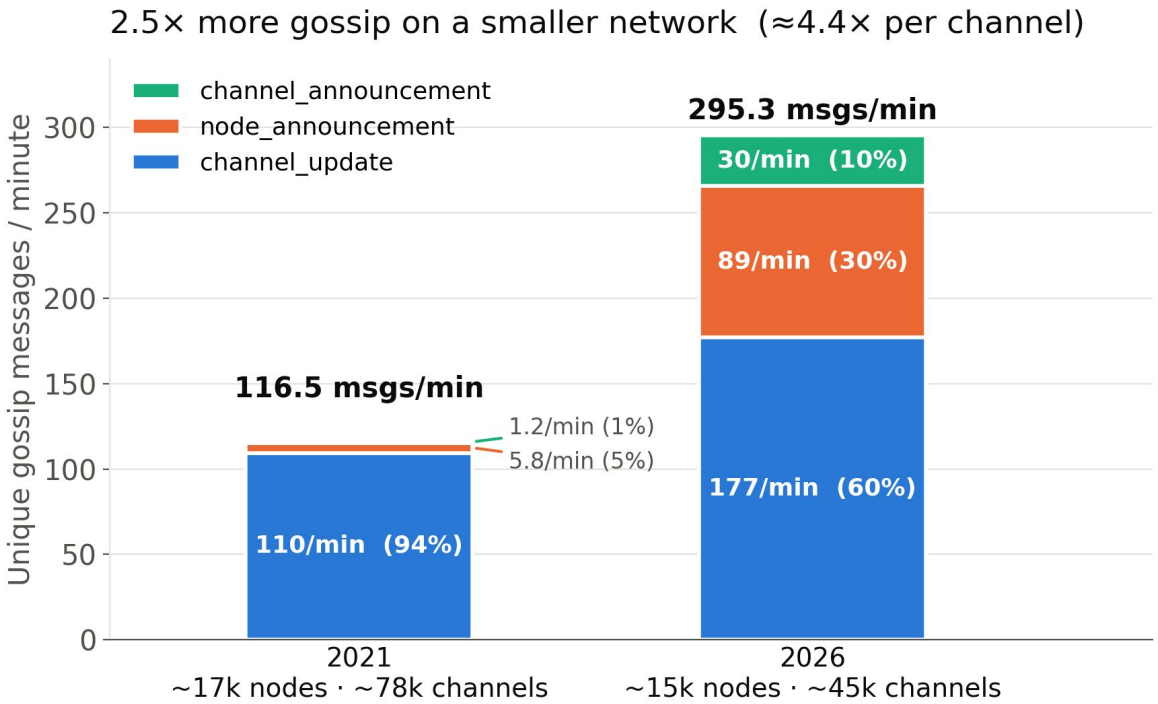
Current network status: Methodology

- Custom fork of ldk-node
- Proportional, rotating selection of peer nodes
- Store the sending peer + receipt timestamp

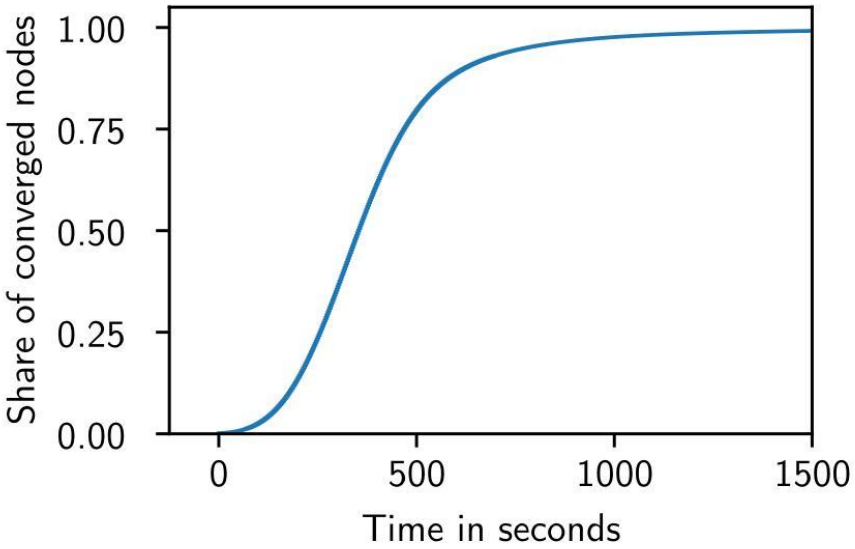
Current network status: Defaults

- Core Lightning default # of gossip syncers: 5 -> 10
- LND & Core Lightning have outbound B/W limits of 1 MB/s
- Rotate peers based on “quality” of received gossip
- Rust-lightning will request last 2 weeks from the first 5 peers
- Eclair syncs from 5 peers, ranked by channel capacity; no random peers

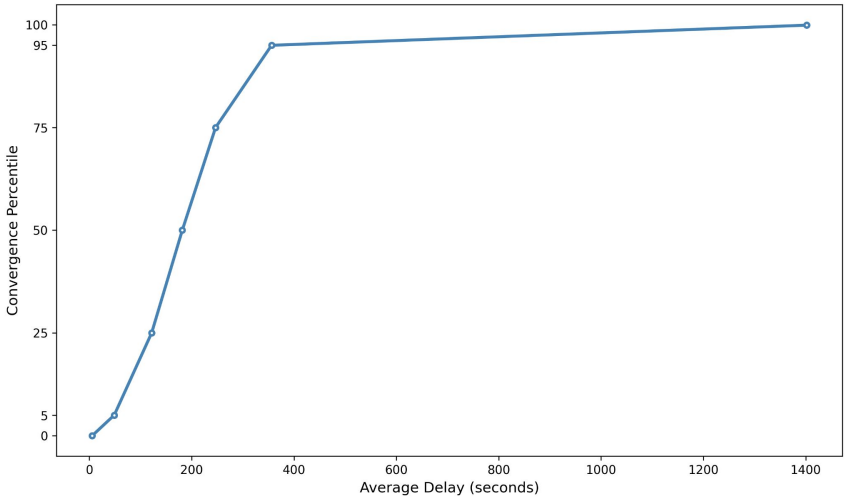
Current network status



Current network status: Propagation Delay

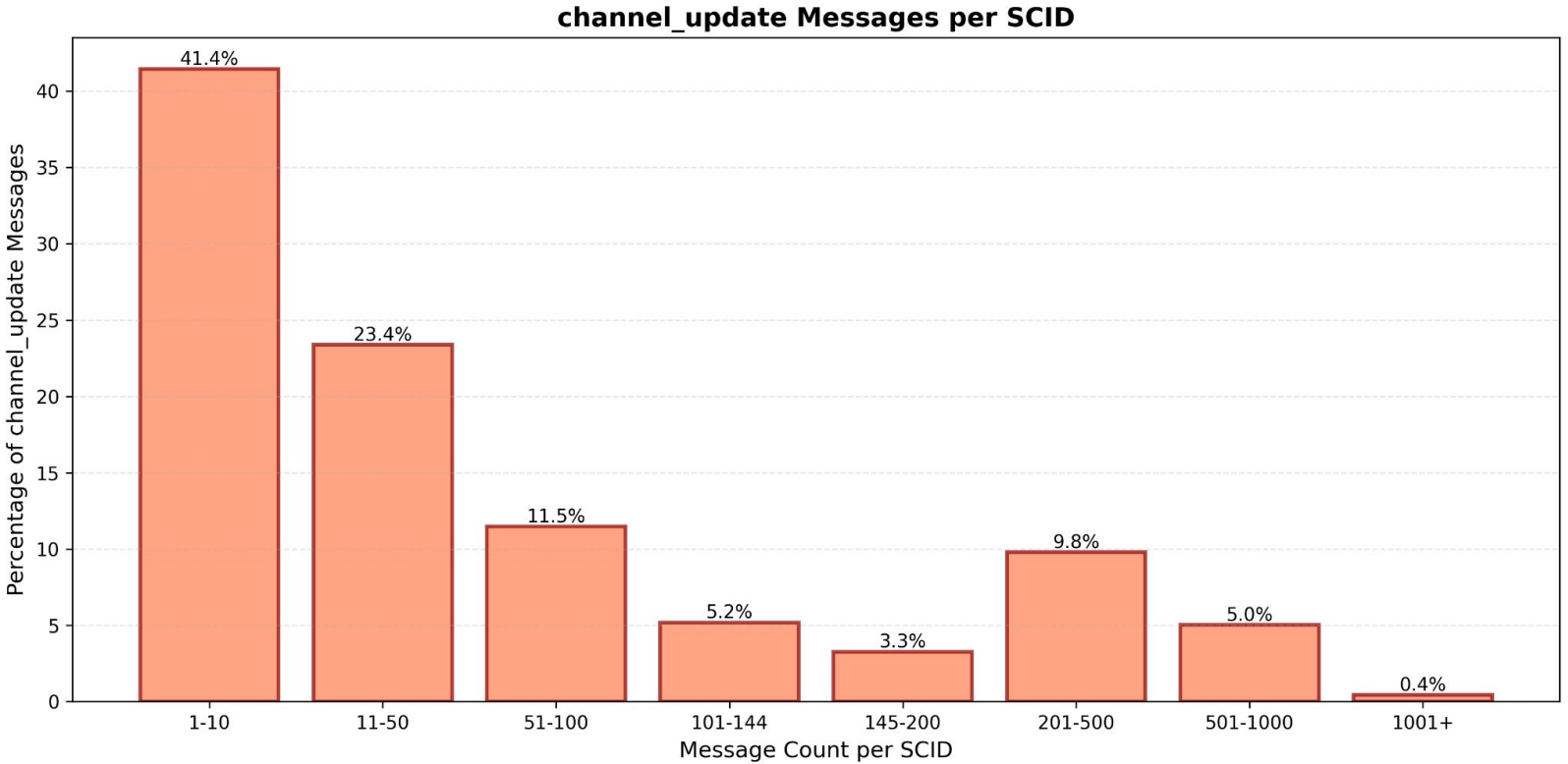


Gogge et al., 2022

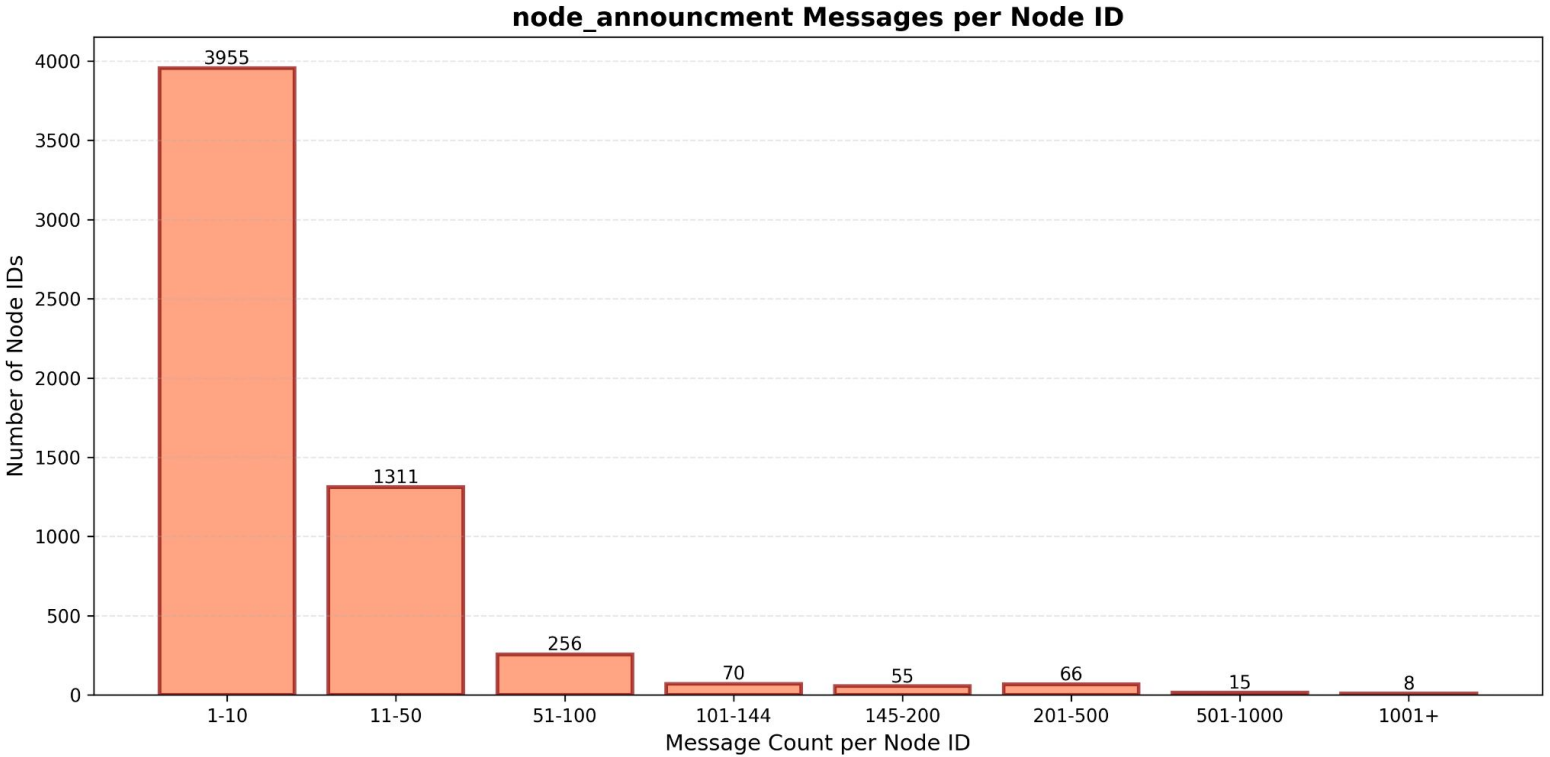


This work, 2026

Current network status: Fee updates

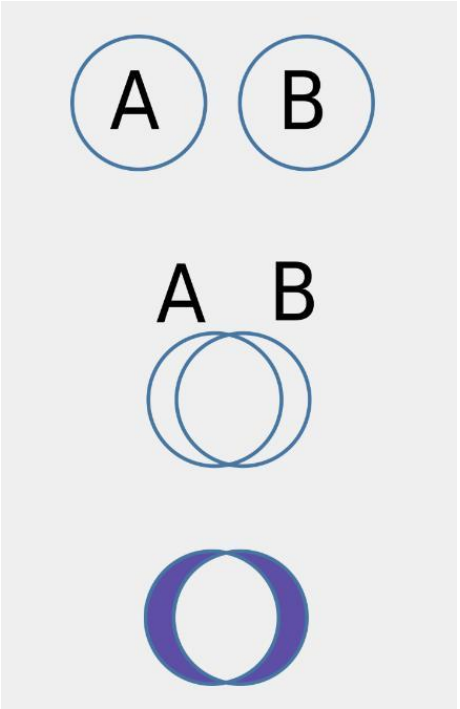


Current network status: Node announcements



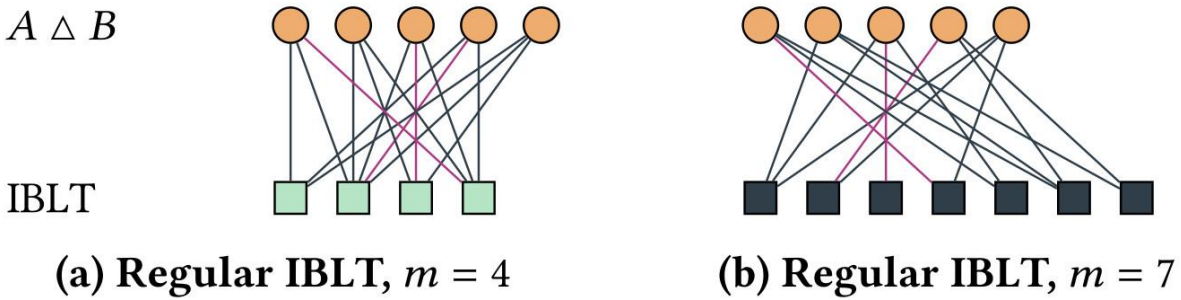
Review: Set Reconciliation

- Two (or N) peers
- Count of set elements, **n**
- Count of differences, **d**
- PDAs, PGP keyservers
- Nostr relays
- CRDTs (multi-editor documents)



Review: IBLT

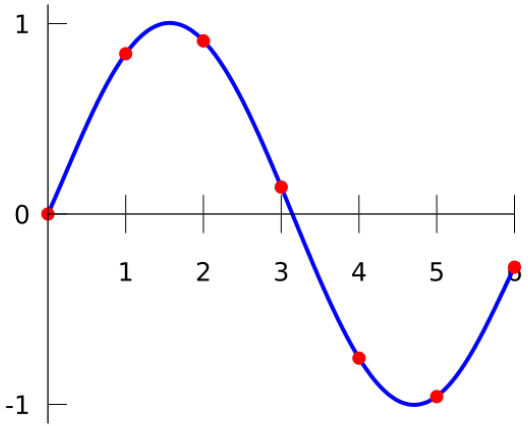
- O(n) to create and decode
- Overhead for set element -> symbol
- Overhead in # of symbols for P(decode)
- Linear; combine with XOR



(a) Regular IBLT, $m = 4$ **(b) Regular IBLT, $m = 7$**

Review: PinSketch (Minisketch)

- O(n) to create, O(d^2) to decode
- Sketch capacity **c** limits difference size
- Sketch elements are **b** bits; size is **b*c**
- Operations scale O(**b**) with HW accel.
- Linear; combine with XOR



Review: PinSketch (Minisketch)

Think BCH codes, error correction, and GF(2^b)

Alice = [1, 2, 3] Bob = [1, 2, 3, 4]

Capacity = 1, so one syndrome; sums

S_a = 1+2+3 = 6 **S_b** = 1+2+3+4 = 10

Difference = 10-6 = 4

Review: PinSketch (Minisketch)

Think BCH codes, error correction, and GF(2^b)

Alice = [1, 2, 3] Bob = [1, 2, 3, 4]

Capacity = 1, so one syndrome; sums

S_a = 1+2+3 = 6 **S_b** = 1+2+3+4 = 10

Difference = 10-6 = 4

Capacity -> Syndromes -> Linear equations

Solutions -> Special polynomials -> Differences

Alice = [2, 3] Bob = [1, 2, 3, 4]

S_a = [5, 13] **S_b** = [10, 30]

S_b - S_a = [5 = x1+x2, 17 = x1^2+x2^2]

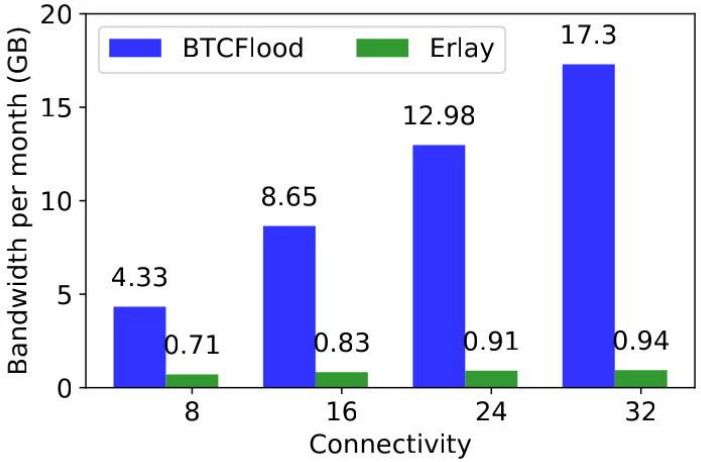
Diff. = [1, 4]

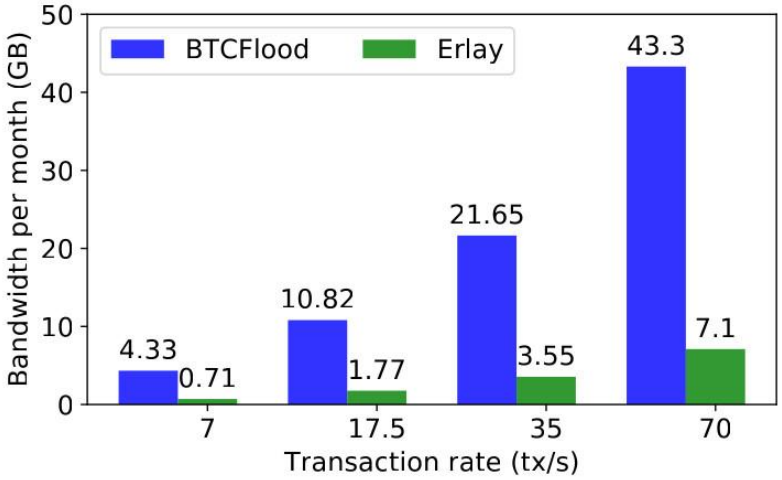
Review: Set Reconciliation

- Many dimensions for tradeoffs
- Topology: Minimum Spanning Tree vs. k-regular
- Universal vs. per-peer state
- IBLT vs. PinSketch vs. Hybrids
- Rateless vs. Estimation
- # of Round Trips

Comparison: Erlay

- Per-peer sketches of salted hashes
- Very short timer; simple estimator





Gossip v1.5

- Type-Length-Value (TLV) format for future changes
- Support new Taproot channel type
- Blockheight-based rate limit
- Optional fields (SPV proofs, etc.)

Gossip v1.5: Proposal Goals

- Lower bandwidth, Simple
- Universal sketches -> Better scaling
- Predictable behavior on a Pi 5
- “Rough consensus” from implementers

Gossip v1.5: Proposal

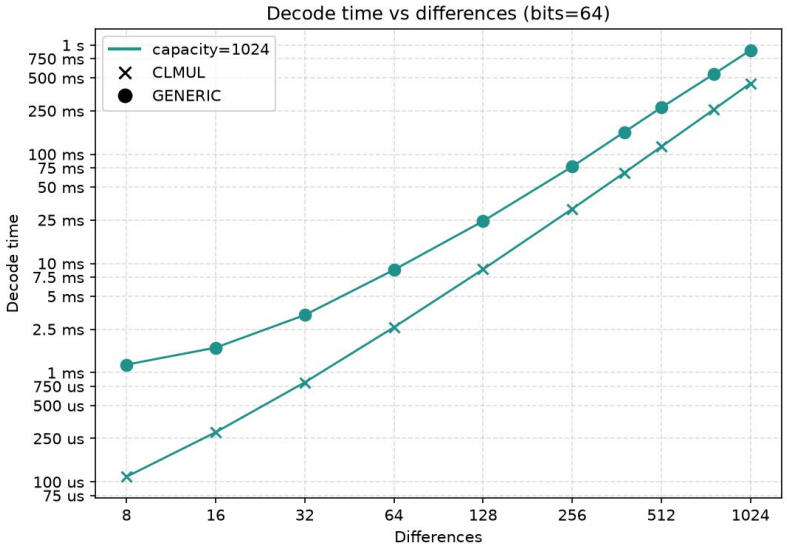
- Sketch per message type
- SCID || Blockheight -> 64 bits! HW friendly
- Internal sketch size >= 1024 elements

Gossip v1.5: Proposal

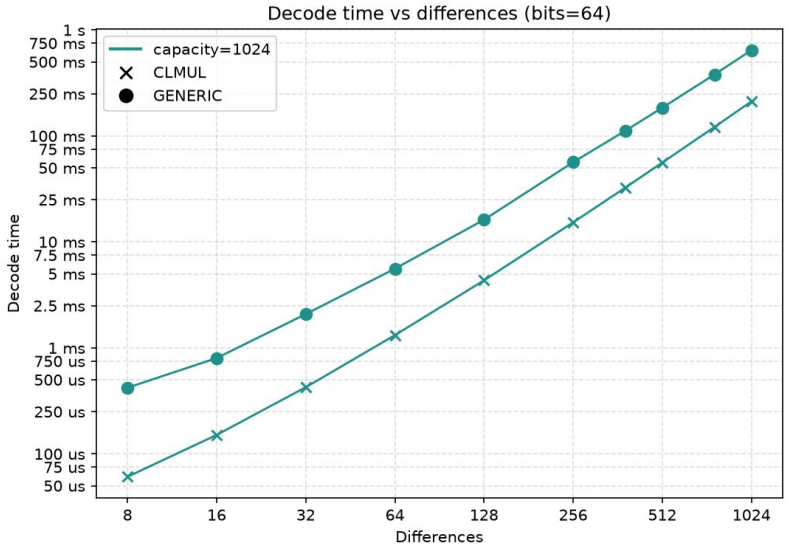
- Sketch per message type
- SCID || Blockheight -> 64 bits! HW friendly
- Internal sketch size >= 1024 elements
- Simple estimator for **d** + single retry
- Query msgs. + flood your own new messages
- 30-60 second timer

Gossip v1.5: Proposal

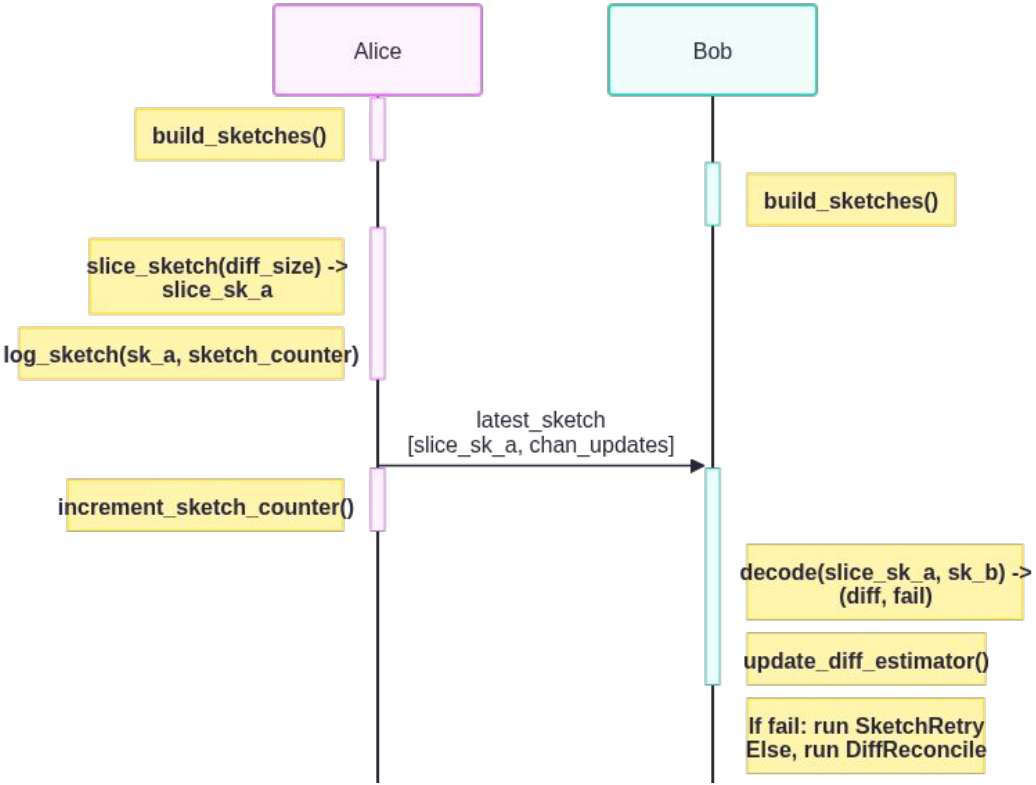
Raspberry Pi 5



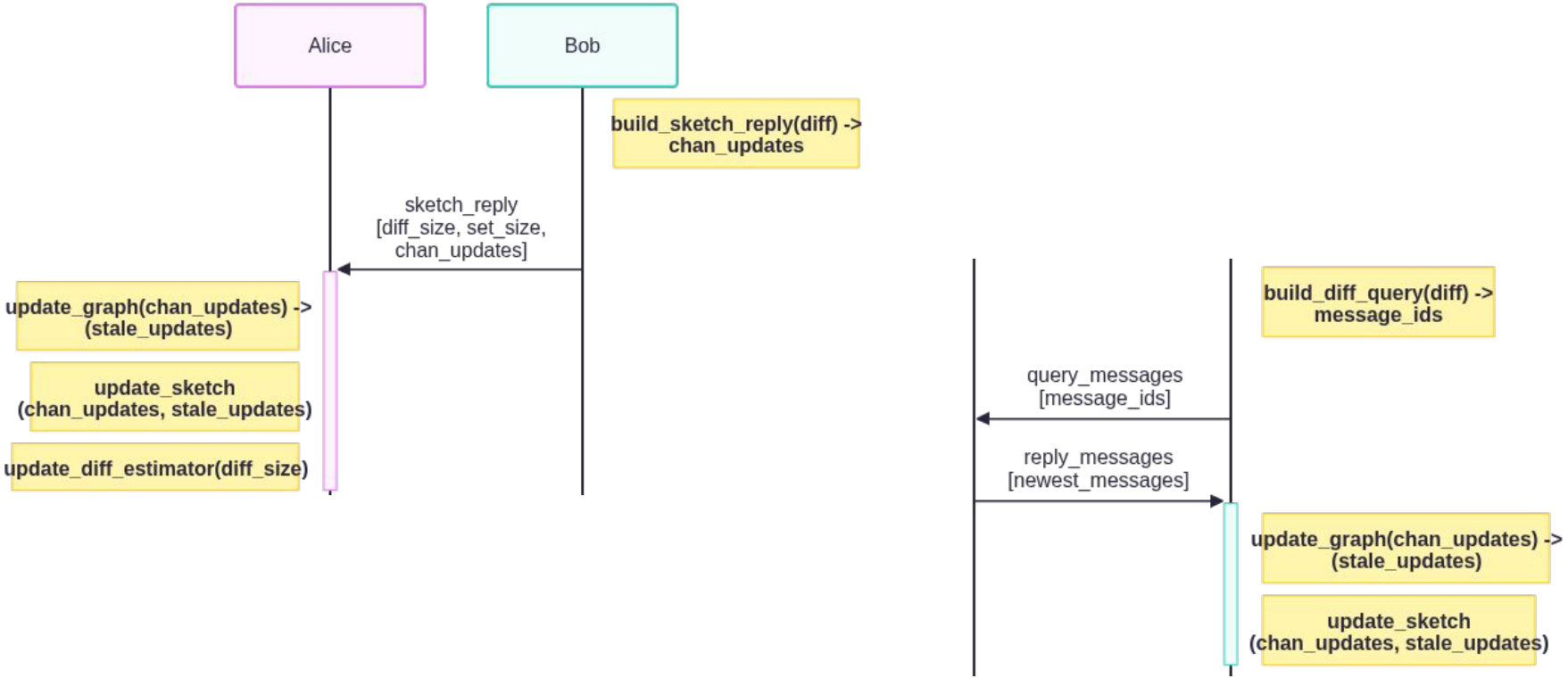
i7 4770 (10 years old)



Gossip v1.5: Proposal

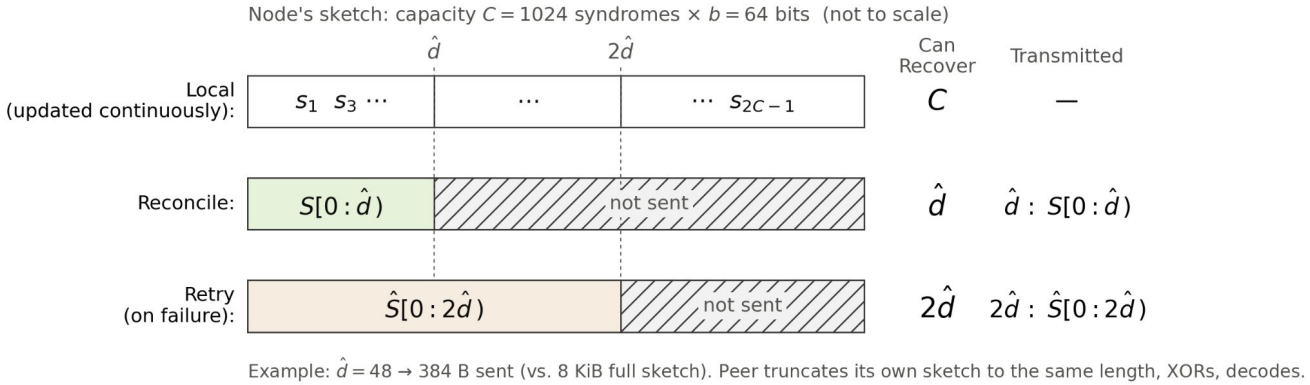


Gossip v1.5: Proposal



Gossip v1.5: Proposal

- Set & sketch update continuously
- Best case: 2 round-trips, 1 sketch + 1 GETDATA msg.
- Worst case: 2 round-trips, 2 sketches, overestimate of **d**



Gossip v1.5: Proposal

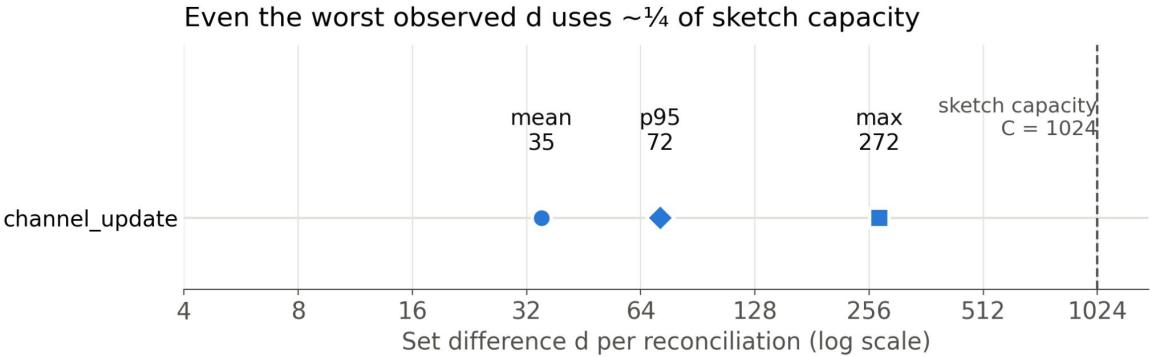
- Given current traffic, what is **d**? Does this work?

Gossip v1.5: Proposal

- Given current traffic, what is **d**? Does this work?
- Simulate! Real graph + Real traffic + model P2P network
- Discrete Event Simulator (DES) -> replay traffic, tweak & measure

Gossip v1.5: Proposal

- Given current traffic, what is **d**? Does this work?
- Simulate! Real graph + Real traffic + model P2P network
- Discrete Event Simulator (DES) -> replay traffic, tweak & measure
- Node degree **p**: [3, 21], mean of 7.5, 60 sec. timer, flood + full reconcile



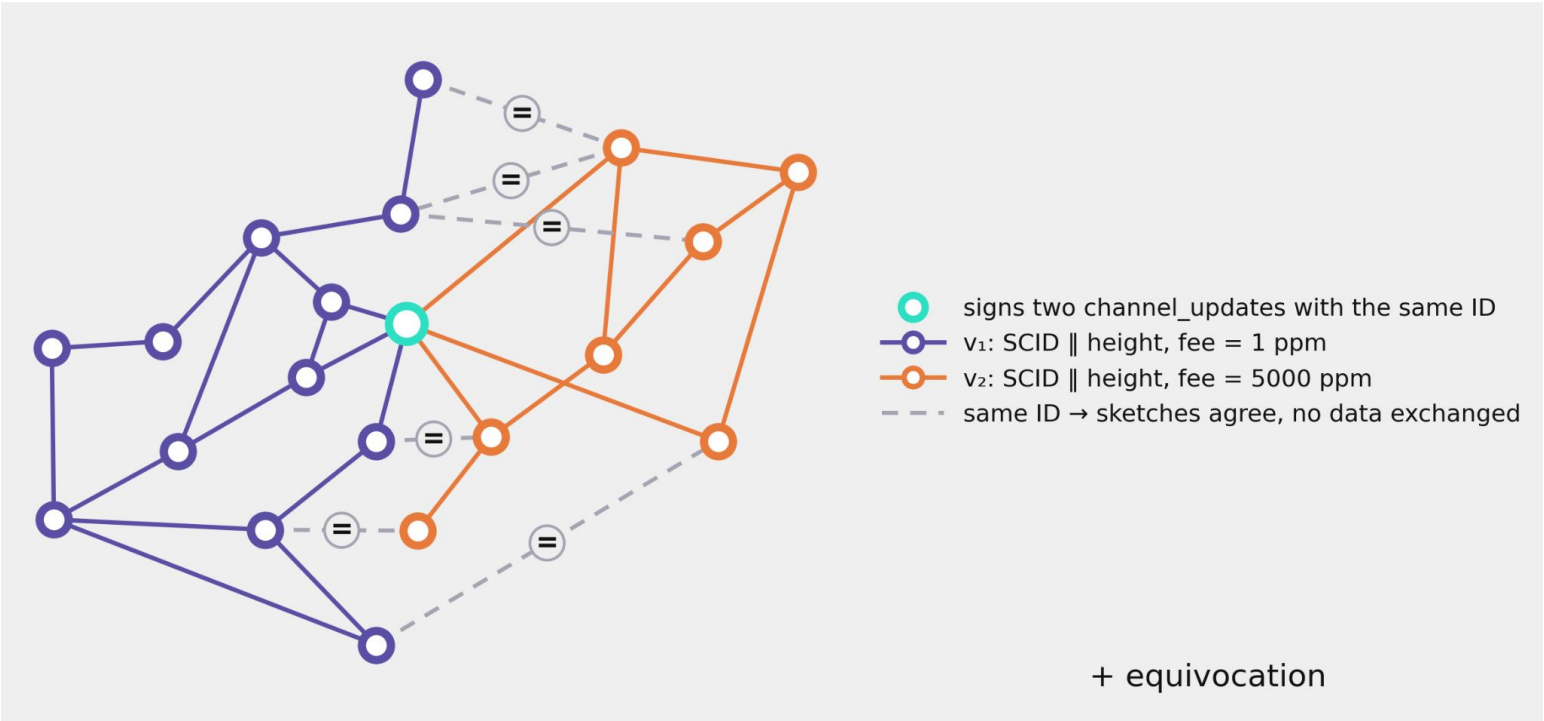
Gossip v1.5: Proposal

- Overhead of flooding scales with **p**; 7.55 with 8 peers
- Sketch overhead is < 2 with oversized sketches
- Missing; dynamic **d,** link latency, peer rotation, intermittent failures
- Specific worst-case scenarios; when will it fail?

Known Problems

- What can an adversary do now? DoS? Privacy? Latency?
- Junk sketch -> waste peer CPU time
- Messages are already public
- Similar to L1: Eclipse? Partitioning? Timing?

Known Problems



Known Problems

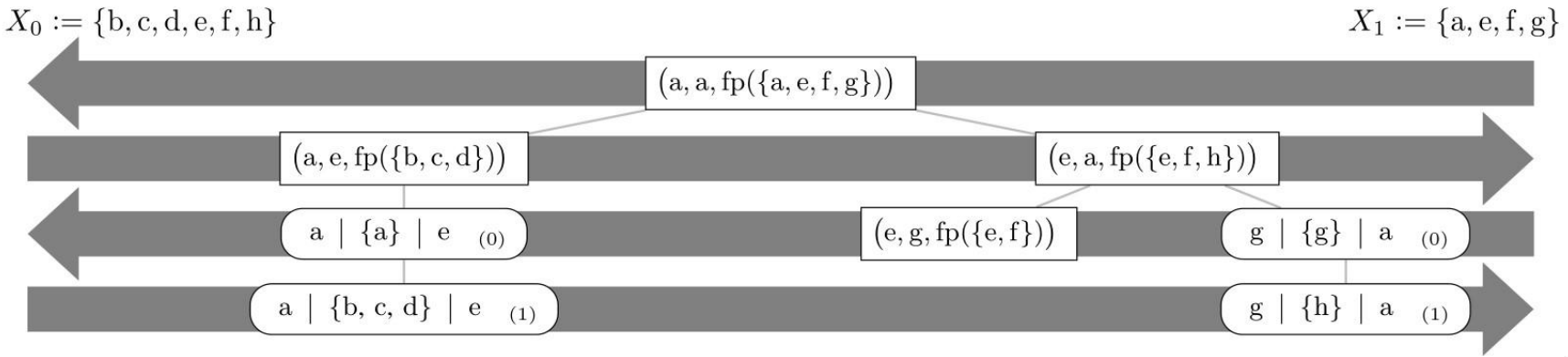
- Use SCID || Blockheight || H(message) -> Back to IBLTs
- Sketch of hashes -> 64 bit elements is too short
- Trees? Bisection? Accumulators? Xor of hashes?

Known Problems

- Use SCID || Blockheight || H(message) -> Back to IBLTs
- Sketch of hashes -> 64 bit elements is too short
- Trees? Bisection? Accumulators? Xor of hashes?
- We can tolerate more rounds, but ideally not more overhead.
- Adversaries can collide hash prefixes / messages

Known Problems

- Range-Based Set Reconciliation (RBSR): Fingerprint, recurse, then handle
- Check intermittently
- New msgs. to ban nodes and/or channels



Future Work

- Richer simulation
- Revisit Minisketch algorithms & perf.
- Chase down misbehaving nodes

Related Work

- Data analysis: 120+ GiB captured over 9 months
- Anonymous credentials: Can we stop doxxing UTXOs?
- Active monitoring & anomaly detection

Related Work

Data:

https://zenodo.org/records/23111326

https://github.com/jharveyb/gossip_observer

https://gossip-observer.jharveyb.xyz/

Draft BOLT:

https://github.com/jharveyb/bolts/tree/gossip_minisketch_draft

More datasets:

https://bnoc.xyz/t/wiki-resources-for-monitoring-and-network-data/37

Thank you!

@jonhbit / jharveyb / lngossip@jharveyb.xyz